

Documentação - Painel de Análise de Sobrevivência Oncológica

1 Introdução

Este documento descreve o processo de tratamento e filtragem de dados oncológicos que são utilizados no Painel de Análise de Sobrevivência Oncológica. As análises podem ser acessadas através do painel disponível em: https://esleycaminhasferreira.shinyapps.io/painel_sobrevivencia/.

2 Sobre os dados

Os dados utilizados foram obtidos a partir do arquivo `pacigeral.dbf`, proveniente da Fundação Oncocentro de São Paulo (FOSP), instituição de referência vinculada à Secretaria de Estado de São Paulo. A FOSP é uma das principais fontes de dados oncológicos no Brasil, mantendo registros hospitalares de grande relevância para a pesquisa e o desenvolvimento de políticas públicas de combate ao câncer no país.

Foram incluídas neoplasias das seguintes categorias:

- C50: Mama
- C51-C58: Órgãos genitais femininos
- C60-C63: Órgãos genitais masculinos
- C64-C68: Trato urinário
- C69-C72: Olho, cérebro e SNC
- C73-C75: Tiróide e outras glândulas
- C76: Outras localizações mal definidas
- C77: Linfonodos
- C80: Localização primária desconhecida

3 Pré-processamento dos dados

Variáveis da base `pacigeral.dbf` que foram utilizadas:

- TOPOGRUP: Grupo da topografia (C50-C80)
- SEXO: Sexo do paciente (1-Masculino, 2-Feminino)
- IDADE: Idade do paciente em anos
- FAIXAETAR: Faixa etária do paciente
- EC: Estádio clínico (classificação TNM)
- ULTINFO: Última informação sobre o paciente
- TRATAMENTO: Código de combinação dos tratamentos realizados
- DTCONSULT: Data da 1ª consulta
- DTDIAG: Data do diagnóstico
- DTTRAT: Data de início do tratamento
- DTULTINFO: Data da última informação do paciente

Variáveis criadas durante o pré-processamento:

- CONSDIAG: Diferença em dias entre consulta e diagnóstico
- TRATCONS: Diferença em dias entre consulta e tratamento
- DIAGTRAT: Diferença em dias entre diagnóstico e tratamento
- TEMPO_OBS_: Tempos de observação calculados em diferentes unidades (dias, meses, trimestres, anos)
- DESFECHO: Variável binária para análise de sobrevivência
- TOPOGRUP_GRUPO: Agrupamento personalizado das topografias
- IDADE_ESTRATIFICADA: Categorização dinâmica da idade usando método de maxstat

Estas variáveis permitem realizar análises completas de sobrevivência (Kaplan-Meier), estimativas de hazard, e modelagem de Cox proporcional, além de fornecer filtros dinâmicos para exploração interativa dos dados oncológicos.

Para o pré-processamento os seguintes pacotes foram utilizados.

```
# Pré-processamento e manipulação de dados
library(foreign)      # Leitura de arquivos .dbf
library(dplyr)        # Manipulação de dados
library(lubridate)    # Manipulação de datas
library(data.table)   # Estrutura de dados eficiente
library(fst)          # Formato de arquivo rápido para armazenamento
library(purrr)        # Programação funcional
```

Script de pré-processamento.

```
## Aqui vamos filtrar as CID's que iremos utilizar

dados_cancer <- read.dbf("data/pacigeral.dbf") # disponível no classroom

# Neoplasias Urogenitais, Urinárias, Olho e Sistema Nervoso Central,
# Endócrinas e Secundárias/Mal Definidas

# C50 a C80 (incluindo todas as CIDs neste intervalo)

grupo_cids <- paste0("C",50:80)
grupo_cids
## Filtrando cids
dados_cancer_filtrado1 <- dados_cancer |>
  filter(TOPOGRUP %in% grupo_cids)

## Filtrando variáveis
dados_cancer_filtrado2 <- dados_cancer_filtrado1 |>
  select(
    TOPOGRUP, #Grupo da topografia (Seleção)
    SEXO, #Sexo do paciente (KM)
    IDADE, #Candidata a maxstat
    FAIXAETAR, #Faixa etaria do paciente (KM)
    EC, #Estadio clinico (KM)

    #Candidatas para o uso do maxstat
    CONSDIAG, #Diferença em dias entre as datas de consulta o diagnostico
    TRATCONS, #Diferença em dias entre as datas de consulta e tratamento
    DIAGTRAT, #Diferença em dias entra as datas de tratamento e diagnostico
```

```

DTCONSULT, #Data da 1a consulta
DTDIAG, #Data do diagnostico

DTTRAT, #Data de inicio do tratamento (Só para checar não entra na conta)
NAOTRAT, #Código da razão para não realização do tratamento
        #(Só para checar não entra na conta)

TRATAMENTO, #Código de combinação dos tratamentos realizados

DTULTINFO, #Data da ultima informacao do paciente

#Sobre desfecho
ULTINFO, #Ultima informacao do paciente
) |>

```

```

mutate(
  TOPOGRUP_GRUPO = factor(
    case_when(
      TOPOGRUP == "C50" ~ "C50 Mama",
      TOPOGRUP %in% c("C51", "C52", "C53", "C54", "C55", "C56", "C57", "C58") ~
        "C51-C58 Órgãos genitais femininos",
      TOPOGRUP %in% c("C60", "C61", "C62", "C63") ~
        "C60-C63 Órgãos genitais masculinos",
      TOPOGRUP %in% c("C64", "C65", "C66", "C67", "C68") ~
        "C64-C68 Trato urinário",
      TOPOGRUP %in% c("C69", "C70", "C71", "C72") ~
        "C69-C72 Olho, cérebro e outras partes do SNC",
      TOPOGRUP %in% c("C73", "C74", "C75") ~
        "C73-C75 Tiróide e outras glândulas",
      TOPOGRUP == "C76" ~
        "C76 Out. localizações e localizações mal definidas",
      TOPOGRUP == "C77" ~
        "C77 Linfonodos",
      TOPOGRUP == "C80" ~
        "C80 Localização primária desconhecida",
    )),

  SEXO = factor(
    case_when(
      SEXO == 1 ~ "Masculino",
      SEXO == 2 ~ "Feminino"
    )),

  FAIXAETAR = factor(
    case_when(
      FAIXAETAR == "00-09" ~ "0 a 9 anos",
      FAIXAETAR == "10-19" ~ "10 a 19 anos",
      FAIXAETAR == "20-29" ~ "20 a 29 anos",
      FAIXAETAR == "30-39" ~ "30 a 39 anos",
      FAIXAETAR == "40-49" ~ "40 a 49 anos",
      FAIXAETAR == "50-59" ~ "50 a 59 anos",
      FAIXAETAR == "60-69" ~ "60 a 69 anos",
      FAIXAETAR == "70+" ~ "70 anos ou mais"
    )
  )
)

```

```

),
levels = c(
  "0 a 9 anos",
  "10 a 19 anos",
  "20 a 29 anos",
  "30 a 39 anos",
  "40 a 49 anos",
  "50 a 59 anos",
  "60 a 69 anos",
  "70 anos ou mais"
),
ordered = TRUE
),

DTULTINFO = dmy(as.character(DTULTINFO)),

#Tempo observado
TEMPO_OBS_DIAG_DIAS = as.numeric(DTULTINFO - DTDIAG),
TEMPO_OBS_CONSULT_DIAS = as.numeric(DTULTINFO - DTCONSULT),

TEMPO_OBS_DIAG_MESES = floor(as.numeric(DTULTINFO - DTDIAG) / 30) + 1,
TEMPO_OBS_CONSULT_MESES = floor(as.numeric(DTULTINFO - DTCONSULT) / 30) + 1,

TEMPO_OBS_DIAG_TRI = floor(as.numeric(DTULTINFO - DTDIAG) / 90) + 1,
TEMPO_OBS_CONSULT_TRI = floor(as.numeric(DTULTINFO - DTCONSULT) / 90) + 1,

TEMPO_OBS_DIAG_ANO = floor(as.numeric(DTULTINFO - DTDIAG) / 365) + 1,
TEMPO_OBS_CONSULT_ANO = floor(as.numeric(DTULTINFO - DTCONSULT) / 365) + 1,

#Tratamento
TRATAMENTO = factor(
  case_when(
    TRATAMENTO == "A" ~ "Cirurgia",
    TRATAMENTO == "B" ~ "Radioterapia",
    TRATAMENTO == "C" ~ "Quimio",
    TRATAMENTO == "D" ~ "Cirurgia + Radioterapia",
    TRATAMENTO == "E" ~ "Cirurgia + Quimio",
    TRATAMENTO == "F" ~ "Radioterapia + Quimio",
    TRATAMENTO == "G" ~ "Cirurgia + Radio + Quimio",
    TRATAMENTO == "H" ~ "Cirurgia + Radio + Quimio + Hormonio",
    TRATAMENTO == "I" ~ "Outras combinações de tratamento",
    TRATAMENTO == "J" ~ "Nenhum tratamento realizado"
  ),
  levels = c(
    "Cirurgia",
    "Radioterapia",
    "Quimio",
    "Cirurgia + Radioterapia",
    "Cirurgia + Quimio",
    "Radioterapia + Quimio",
    "Cirurgia + Radio + Quimio",
    "Cirurgia + Radio + Quimio + Hormonio",
    "Outras combinações de tratamento",
  )

```

```

    "Nenhum tratamento realizado"
  ),
  ordered = FALSE
  # Se TRUE, define como variável ordinal (hierarquia entre categorias)
),

#Estagio da doenca
GRUPO_EC = factor(
  case_when(
    EC %in% c("O", "OA", "OIS", "IS") ~ "Estágio 0",
    EC %in% c("I", "IA", "IA1", "IA2", "IB", "IB1", "IB2", "IC") ~ "Estágio I",
    EC %in% c("II", "IIA", "IIA1", "IIA2", "IIB", "IIC") ~ "Estágio II",
    EC %in% c("III", "IIIA", "IIIB", "IIIC", "IIIC1", "IIIC2") ~ "Estágio III",
    EC %in% c("IV", "IVA", "IVB", "IVC") ~ "Estágio IV",
    EC == "X" ~ "X",
    EC == "Y" ~ "Y"
  )),

# 1 - VIVO, COM CÂNCER
# 2 - VIVO, SOE
# 3 - ÓBITO POR CANCER
# 4 - ÓBITO POR OUTRAS CAUSAS, SOE

ULTINFO = factor(
  case_when(
    ULTINFO == 1 ~ "Vivo, com câncer",
    ULTINFO == 2 ~ "Vivo, sem outra especificação",
    ULTINFO == 3 ~ "Óbito por câncer",
    ULTINFO == 4 ~ "Óbito por outras causas, sem outra especificação"
  )),

DESFECHO = case_when(
  ULTINFO %in% c("Vivo, com câncer",
                 "Vivo, sem outra especificação",
                 "Óbito por outras causas, sem outra especificação") ~ "0",
  ULTINFO == "Óbito por câncer" ~ "1"
),
NAOTRAT = as.factor(NAOTRAT),
DESFECHO = as.numeric(DESFECHO)
)

## Salvando como data.table + FST
dados_cancer_dt <- as.data.table(dados_cancer_filtrado2)
write_fst(dados_cancer_dt, "data/dados_cancer_filtrado.fst")

```

3.1 Construção do painel

Para a construção do painel os seguintes pacotes foram utilizados.

```

# Construção do painel Shiny
library(shiny)           # Framework principal
library(shinydashboard) # Layout de dashboard
library(shinycssloaders) # Indicadores de carregamento

```

```
library(shinyWidgets)  # Widgets adicionais

# Visualização e análise
library(highcharter)   # Gráficos interativos
library(survival)      # Análise de sobrevivência
library(survminer)     # Visualização de análise de sobrevivência
library(muhaz)         # Estimção de hazard
library(broom)         # Tidy de modelos estatísticos
library(DT)           # Tabelas interativas
```

O código-fonte completo do painel está disponível no repositório GitHub: https://github.com/EsleyCaminhas/painel_sobrevivencia

3.1.1 Implementação do ponto de corte para a variável idade

Dentro do painel, foi implementada uma funcionalidade para determinar automaticamente o ponto de corte ótimo da idade usando o método de maxstat, que se adapta dinamicamente às seleções de tempo do usuário:

```
# 1. Criar uma expressão reativa que depende das seleções de tempo do usuário
# Usaremos os inputs da aba Kaplan-Meier (input$Tempo_int e input$len_tempo)
# para controlar o cálculo
dados_cancer_reativo <- reactive({

  # Garante que os inputs de tempo existem antes de prosseguir
  req(input$Tempo_int, input$len_tempo)

  # Cria o nome da coluna de tempo dinamicamente com base na seleção do usuário
  tempo_selecionado <- paste0("TEMPO_OBS_", input$Tempo_int, "_", input$len_tempo)

  # Filtra o data.frame para garantir que o tempo seja maior que zero na escala
  # selecionada
  dados_filtrados_tempo <- dados_cancer_base |>
    filter(.data[[tempo_selecionado]] > 0)

  # 2. Encontrar o ponto de corte ótimo para a idade usando o tempo dinâmico
  res_cutpoint <- surv_cutpoint(
    data = dados_filtrados_tempo,
    time = tempo_selecionado, # Usa a coluna de tempo dinâmica
    event = "DESFECHO",
    variables = "IDADE",
    minprop = 0.2
  )

  # 3. Extrair o valor numérico do ponto de corte
  ponto_de_corte <- res_cutpoint$cutpoint$cutpoint

  # 4. Criar as etiquetas descritivas
  label_menor <- paste0("Idade menor que ", ponto_de_corte, " anos")
  label_maior <- paste0("Idade maior ou igual a ", ponto_de_corte, " anos")

  # 5. Adicionar a nova coluna e renomear os níveis do fator ao data.frame
  # filtrado
```

```

dados_com_idade_estratificada <- dados_filtrados_tempo |>
  mutate(
    IDADE_ESTRATIFICADA = surv_categorize(res_cutpoint)$IDADE,
    IDADE_ESTRATIFICADA = recode_factor(IDADE_ESTRATIFICADA,
                                         "low" = label_menor,
                                         "high" = label_maior)
  )

  return(dados_com_idade_estratificada)
})

```

Esta implementação permite que o ponto de corte da idade seja determinado dinamicamente com base na unidade de tempo selecionada pelo usuário (dias, meses, trimestres ou anos), garantindo que a estratificação seja sempre otimizada para a janela temporal em questão.

3.1.2 Dados implausíveis

Durante a criação da variável TEMPO_OBS_DIAG_MESES, foram observados valores negativos, indicando inconsistências temporais nos dados (como datas de última informação anteriores às datas de diagnóstico). Esses registros foram removidos para garantir a qualidade da análise:

```

dados_cancer_base <- read_fst("data/dados_cancer_filtrado.fst",
                             as.data.table = TRUE) |>
  filter(TEMPO_OBS_DIAG_MESES > 0)

```

4 Anexo

4.0.1 global.R

```

# Construção do painel Shiny

## global.R
# Bibliotecas utilizadas em app.R ou ui.R ou server.R

library(shiny)

library(shinydashboard)
library(shinycssloaders)
library(shinyWidgets)

library(highcharter)
library(fst)

library(dplyr)

library(survival)
library(survminer)

library(DT)

```

```
library(purrr)
library(muhaz)

library(broom)
```

4.0.2 ui.R

```
## ui.R

ui <- dashboardPage(

  skin = "purple",

  #Cabecalho
  dashboardHeader(
    title = strong("Painel ASO"),
    tags$li(class = "dropdown",
             tags$style(".justified-text { text-align: justify; }"))
  ),

  #Barra lateral
  dashboardSidebar(
    sidebarMenu(
      menuItem("Sobre o Painei",
               tabName = "Info"),
      menuItem("Análise das Variáveis",
               tabName = "Graphs"),
      menuItem("Curvas de Kaplan-Meier",
               tabName = "KM"),
      menuItem("Função de Risco",
               tabName = "Hazard"),
      menuItem("Modelo de Cox",
               tabName = "COX")
    )
  ),

  #Corpo principal
  dashboardBody(

    tags$style("justified-text { text-align: justify; }"),

    tabItems(

      #Secao com as informacoes
      tabItem(tabName = "Info",

               h2(strong("Painel de Análise de Sobrevivência Oncológica (ASO)")),

               hr(style = "border: 1px solid #ccc; margin: 20px 0;"),

               p("Seja bem vinda(o)!"),
```



```

p(
  class = "justified-text",
  "Este painel foi desenvolvido como parte do seminário apresentado à disciplina de Análise de Dados em Saúde. Seu objetivo é permitir a exploração e a análise visual de dados de sobrevivência de pacientes com câncer no Brasil.",
),

p(
  class = "justified-text",
  "A FOSP é uma das principais fontes de dados oncológicos no Brasil, mantendo registros de dados de sobrevivência de pacientes com câncer desde 1978.",
),

h3(strong("Sobre os Dados")),

p(
  class = "justified-text",
  "Os dados utilizados neste painel compreendem um subconjunto específico de tipos de câncer registrados no Brasil, incluindo:",
),

tags$ul(
  tags$li("C50: Mama"),
  tags$li("C51-C58: Órgãos genitais femininos"),
  tags$li("C60-C63: Órgãos genitais masculinos"),
  tags$li("C64-C68: Trato urinário"),
  tags$li("C69-C72: Olho, cérebro e outras partes do SNC"),
  tags$li("C73-C75: Tiróide e outras glândulas"),
  tags$li("C76: Outras localizações e localizações mal definidas"),
  tags$li("C77: Linfonodos"),
  tags$li("C80: Localização primária desconhecida")
),

br(),

div(
  style = "display: flex; align-items: center; gap: 10px;",

  tags$img(src = "images/fosp.png", height = "40px", alt = "Logo FOSP"),

  p(
    class = "justified-text",
    style = "margin: 0;", # Remove margem padrão do parágrafo
    "Para mais informações sobre a FOSP visite o site:",
    a("https://fosp.saude.sp.gov.br/", href = "https://fosp.saude.sp.gov.br/")
  )
),

h3(strong("Documentação")),

actionButton("generate", "Gerar pdf da documentação"),
uiOutput("pdfview"),

hr(style = "border: 1px solid #ccc; margin: 20px 0;"),

div(

```

```

        style = "display: flex; justify-content: center; align-items: center; gap: 30px; padding: 10px; margin: 10px 0;">
        tags$img(src = "images/dest.png", height = "100px", alt = "Logo DEST", style = "object-fit: cover; width: 100px;")
        tags$img(src = "images/ufes.png", height = "100px", alt = "Logo UFES", style = "object-fit: cover; width: 100px;")
    ),
    #Secao com os graficos
    tabItem(tabName = "Graphs",

        sidebarLayout(
            sidebarPanel(

                h4(strong("Filtros para grupo e variável")),
                hr(),
                width = 3,

                pickerInput(
                    inputId = "grupo_cid_1",
                    label = "Selecione o grupo (topografia):",
                    choices = c("C50 Mama" = "C50 Mama", "C51-C58 Órgãos genitais femininos" = "C51-C58 Órgãos genitais femininos", "C60-C63 Órgãos genitais masculinos" = "C60-C63 Órgãos genitais masculinos"),
                    selected = c("C50 Mama" = "C50 Mama", "C51-C58 Órgãos genitais femininos" = "C51-C58 Órgãos genitais femininos", "C60-C63 Órgãos genitais masculinos" = "C60-C63 Órgãos genitais masculinos"),
                    multiple = TRUE,
                    options = list(`actions-box` = TRUE, `selected-text-format` = "count > 8", `count-suffix` = "de"),
                ),

                selectInput(
                    inputId = "variavel_1",
                    label = "Selecione a variável:",
                    choices = c("Faixa etária" = "FAIXAETAR", "Idade (ponto de corte)" = "IDADE_ESTRATIFICADA", "Sexo" = "SEXO"),
                    selected = "IDADE_ESTRATIFICADA"
                )
            ),

            mainPanel(
                uiOutput("alert_box1"),
                withSpinner(highchartOutput("grafico_barras"), type = 6)
            )
        ),

    #Secao com as curvas de KM
    tabItem(tabName = "KM",
        sidebarLayout(
            sidebarPanel(

                h4(strong("Filtros para grupo e variável")),
                hr(),
                width = 3,

```

```

pickerInput(
  inputId = "grupo_cid_2",
  label = "Selecione o grupo (topografia):",
  choices = c("C50 Mama" = "C50 Mama", "C51-C58 Órgãos genitais femininos" = "C51-C58",
  selected = c("C50 Mama"
               # "C51-C58 Órgãos genitais femininos", "C60-C63 Órgãos genitais masculinos"
  ),
  multiple = TRUE,
  options = list(`actions-box` = TRUE, `selected-text-format` = "count > 8", `count-s
),

selectInput(
  inputId = "km_variable",
  label = "Selecione a variável:",
  choices = c("Faixa etária" = "FAIXAETAR", "Sexo" = "SEX0", "Idade (ponto de corte)"
  selected = "IDADE_ESTRATIFICADA"
),

br(),

h4(strong("Filtros relacionados ao tempo de acompanhamento")),
hr(),

selectInput(
  inputId = "Tempo_int",
  label = "Selecione o inicio do acompanhamento:",
  choices = c("Diagnóstico" = "DIAG", "Consulta" = "CONSULT"),
  selected = "DIAG"
),

selectInput(
  inputId = "len_tempo",
  label = "Selecione a unidade de tempo:",
  choices = c("Dias" = "DIAS", "Meses (30 dias)" = "MESES", "Trimestres (90 dias)" = "
  selected = "DIAS"
),

hr(),

shinyWidgets::materialSwitch(
  inputId = "show_ci",
  label = "Mostrar o intervalo de confiança?",
  value = FALSE,
  status = "info"
),

# ===== INÍCIO DA MODIFICAÇÃO =====
shinyWidgets::materialSwitch(
  inputId = "show_logrank",
  label = "Mostrar teste de Log-Rank?",
  value = FALSE,
  status = "info"
)

```

```

),
mainPanel(
  uiOutput("alert_box2"),
  withSpinner(highchartOutput("km_plot", height = "600px"), type = 6),

  # ===== INÍCIO DA MODIFICAÇÃO =====
  withSpinner(uiOutput("logrank_test_output"), type = 6),
  # ===== FIM DA MODIFICAÇÃO =====

  hr(style = "border: 1px solid #ccc; margin: 20px 0;"),
  withSpinner(uiOutput("tabelas_por_grupo"), type = 6)
)
),
),

#Secao da Função de Risco
tabItem(tabName = "Hazard",
  sidebarLayout(
    sidebarPanel(

      h4(strong("Filtros para grupo e variável")),
      hr(),
      width = 3,

      pickerInput(
        inputId = "grupo_cid_3",
        label = "Selecione o grupo (topografia):",
        choices = c("C50 Mama" = "C50 Mama", "C51-C58 Órgãos genitais femininos" = "C51-C58"),
        selected = c("C50 Mama"),
        multiple = TRUE,
        options = list(`actions-box` = TRUE, `selected-text-format` = "count > 8", `count-s
      ),

      selectInput(
        inputId = "hazard_variable",
        label = "Selecione a variável:",
        # ===== INÍCIO DA CORREÇÃO =====
        choices = c("Sexo" = "SEXO", "Faixa etária" = "FAIXAETAR", "Idade (ponto de corte)" = "IDADE_ESTRATIFICADA"),
        # ===== FIM DA CORREÇÃO =====
        selected = "IDADE_ESTRATIFICADA"
      ),

      br(),

      h4(strong("Filtros relacionados ao tempo de acompanhamento")),
      hr(),

      selectInput(
        inputId = "Tempo_int2",
        label = "Selecione o inicio do acompanhamento:",
        choices = c("Diagnóstico" = "DIAG", "Consulta" = "CONSULT"),
        selected = "DIAG"
      )
    )
  )
)

```

```

    ),

    selectInput(
      inputId = "len_tempo2",
      label = "Selecione a unidade de tempo:",
      choices = c("Dias" = "DIAS", "Meses (30 dias)" = "MESES", "Trimestres (90 dias)" = "TRIMESTRES"),
      selected = "DIAS"
    )
  ),
  mainPanel(
    uiOutput("alert_box3"),
    withSpinner(highchartOutput("hazard_plot", height = "600px"), type = 6)
  )
),

# Aba do Modelo de Cox
tabItem(tabName = "COX",
  sidebarLayout(
    sidebarPanel(

      h4(strong("Filtros para o modelo")),
      hr(),
      width = 3,

      pickerInput(
        inputId = "grupo_cid_4",
        label = "Selecione o grupo (topografia):",
        choices = c("C50 Mama" = "C50 Mama", "C51-C58 Órgãos genitais femininos" = "C51-C58 Órgãos genitais femininos"),
        selected = c("C50 Mama"),
        multiple = TRUE,
        options = list(`actions-box` = TRUE, `selected-text-format` = "count > 8", `count-suffix` = " ")
      ),

      pickerInput(
        inputId = "cox_variables",
        label = "Selecione as variáveis (covariáveis):",
        choices = c("Faixa etária" = "FAIXAETAR", "Idade (ponto de corte)" = "IDADE_ESTRATIFICADA", "SEXO"),
        selected = c("IDADE_ESTRATIFICADA", "SEXO"),
        multiple = TRUE,
        options = list(`actions-box` = TRUE)
      ),

      br(),

      h4(strong("Filtros de tempo")),
      hr(),

      selectInput(
        inputId = "Tempo_int3",
        label = "Selecione o inicio do acompanhamento:",
        choices = c("Diagnóstico" = "DIAG", "Consulta" = "CONSULT"),
        selected = "DIAG"
      )
    )
  )
)

```


4.0.3 server.R

```
# Carregue todos os pacotes necessários no início
library(shiny)
library(shinyWidgets)
library(fst)
library(dplyr)
library(data.table)
library(highcharter)
library(survival)
library(survminer)
library(DT)
library(broom)
library(purrr)
library(muhaz)
library(shinycssloaders)

# Dica: É uma boa prática usar caminhos relativos para que o projeto funcione em qualquer computador.
# O caminho original "data/dados_cancer_filtrado.fst" é mais portátil.
dados_cancer_base <- read_fst("data/dados_cancer_filtrado.fst",
                             as.data.table = TRUE) |>
  filter(TEMPO_OBS_DIAG_MESES > 0)

## Aqui fica a parte onde trabalhamos com os dados e criamos os outputs
## que serão utilizados na ui

server <- function(input, output, session) {

  # =====
  # ==== INÍCIO DA MODIFICAÇÃO: LÓGICA DE CUTPOINT MOVIDA PARA DENTRO DO SERVER ====
  # =====

  # 1. Criar uma expressão reativa que depende das seleções de tempo do usuário.
  # Usaremos os inputs da aba Kaplan-Meier (input$Tempo_int e input$len_tempo) para controlar o cálculo
  dados_cancer_reativo <- reactive({

    # Garante que os inputs de tempo existem antes de prosseguir
    req(input$Tempo_int, input$len_tempo)

    # Cria o nome da coluna de tempo dinamicamente com base na seleção do usuário
    tempo_selecionado <- paste0("TEMPO_OBS_", input$Tempo_int, "_", input$len_tempo)

    # Filtra o data.frame para garantir que o tempo seja maior que zero na escala selecionada
    dados_filtrados_tempo <- dados_cancer_base |>
      filter(.data[[tempo_selecionado]] > 0)

    # 2. Encontrar o ponto de corte ótimo para a idade usando o tempo dinâmico.
    res_cutpoint <- surv_cutpoint(
      data = dados_filtrados_tempo,
      time = tempo_selecionado, # Usa a coluna de tempo dinâmica
      event = "DESFECHO",
```

```

    variables = "IDADE",
    minprop = 0.2
  )

  # 3. Extrair o valor numérico do ponto de corte.
  ponto_de_corte <- res_cutpoint$cutpoint$cutpoint

  # 4. Criar as etiquetas descritivas.
  label_menor <- paste0("Idade menor que ", ponto_de_corte, " anos")
  label_maior <- paste0("Idade maior ou igual a ", ponto_de_corte, " anos")

  # 5. Adicionar a nova coluna e renomear os níveis do fator ao data.frame filtrado.
  dados_com_idade_estratificada <- dados_filtrados_tempo |>
    mutate(
      IDADE_ESTRATIFICADA = surv_categorize(res_cutpoint)$IDADE,
      IDADE_ESTRATIFICADA = recode_factor(IDADE_ESTRATIFICADA,
                                           "low" = label_menor,
                                           "high" = label_maior)
    )

  # Retorna o dataframe final e reativo
  return(dados_com_idade_estratificada)
})

# =====
# ==== FIM DA MODIFICAÇÃO ====
# =====

#### aba Análises Gráficas

#####

## Histograma

dados_filtrados <- reactive({
  # ALTERADO: Usa dados_cancer_reativo() em vez de dados_cancer
  dados_cancer_reativo() |>
    filter(TOPOGRUP_GRUPO %in% input$grupo_cid_1)
})

output$alert_box1 <- renderUI({
  if(length(input$grupo_cid_1) < 1) {
    div(class = "alert alert-warning",
        icon("exclamation-triangle"),
        "Selecione, pelo menos, uma variável para visualizar o gráfico.")
  }
})

output$grafico_barras <- renderHighchart({
  req(input$grupo_cid_1)

```



```

contagem <- dados_filtrados() |>
  count(.data[[input$variavel_1]])

nome_var <- case_when(input$variavel_1 == "SEXO" ~ "Sexo",
                      input$variavel_1 == "FAIXAETAR" ~ "Faixa etária",
                      input$variavel_1 == "IDADE ESTRATIFICADA" ~ "Idade (ponto de corte)",
                      input$variavel_1 == "GRUPO_EC" ~ "Estádio clínico",
                      input$variavel_1 == "ULTINFO" ~ "Desfecho Tratamento",
                      input$variavel_1 == "TRATAMENTO" ~ "Tratamento")

hchart(contagem, "column",
       hcaes(x = !!sym(input$variavel_1), y = n),
       name = "Número de observações",
       color = "#4682B4") |>
  hc_title(text = paste("Frequência observada para a variável ", nome_var)) |>
  hc_xAxis(title = list(text = nome_var)) |>
  hc_yAxis(max = max(20000, max(contagem$n, na.rm = TRUE)),
          title = list(text = "Número de observações"))
})

#### aba Curvas de Kaplan-Meier

#####

janela_tempo <- reactive({
  case_when(
    input$len_tempo == "DIAS" ~ "Tempo (dias)",
    input$len_tempo == "MESES" ~ "Tempo (meses)",
    input$len_tempo == "TRI" ~ "Tempo (trimestres)",
    input$len_tempo == "ANO" ~ "Tempo (anos)"
  )
})

## Gráfico Kaplan-Meier

dados_filtrados_km <- reactive({
  # ALTERADO: Usa dados_cancer_reativo() em vez de dados_cancer
  dados <- dados_cancer_reativo() |>
    filter(TOPOGRUP_GRUPO %in% input$grupo_cid_2) |>
    select(tempo = paste0("TEMPO_OBS_", input$Tempo_int, "_", input$len_tempo),
           Grupo = input$km_variable,
           DESFECHO
    ) |>
    mutate(tempo = as.numeric(tempo))

  dados
})

output$alert_box2 <- renderUI({
  if(length(input$grupo_cid_2) < 1) {
    div(class = "alert alert-warning",
        icon("exclamation-triangle"),

```

```

      "Selecione, pelo menos, uma variável para visualizar o gráfico.")
    }
  })

output$km_plot <- renderHighchart({

  req(input$grupo_cid_2)

  dados <- dados_filtrados_km()

  fit <- survfit(Surv(tempo, DESFECHO) ~ Grupo, data = dados)

  hchart(fit, type = "line", ranges = input$show_ci) |>
    hc_title(text = "Gráfico de Sobrevivência") |>
    hc_xAxis(title = list(text = janela_tempo())) |>
    hc_yAxis(title = list(text = "Probabilidade de Sobrevivência"),
      labels = list(formatter = JS("function() { return Highcharts.numberFormat(this.value, 3)
    hc_tooltip(
      formatter = JS(paste0("function() {

        if (typeof this.point.low !== 'undefined' && typeof this.point.high !== 'undefined') {

          return '<b> Intervalo </b><br/>' +
            '"", janela_tempo(), ": <b>' + this.x + '</b><br/>' +
            'IC%: <b>' + Highcharts.numberFormat(this.point.low, 3) +
            ' - ' + Highcharts.numberFormat(this.point.high, 3) + '</b>';

        } else {

          return '<b>' + this.series.name + '</b><br/>' +
            '"", janela_tempo(), ": <b>' + this.x + '</b><br/>' +
            'Sobrevivência: <b>' + Highcharts.numberFormat(this.y, 3) + '</b>';

        }
      }"))),
    shared = FALSE,
    crosshairs = TRUE
  ) |>
    hc_legend(align = "center", verticalAlign = "bottom", layout = "horizontal") |>
    hc_plotOptions(series = list(marker = list(radius = 0)))
})

# ===== INÍCIO DA MODIFICAÇÃO =====
output$logrank_test_output <- renderUI({
  # Requer que o switch esteja ativado e que haja filtros selecionados
  req(input$show_logrank, input$grupo_cid_2)

  dados <- dados_filtrados_km()

  # O teste só faz sentido para 2 ou mais grupos
  if (length(unique(dados$Grupo)) < 2) {
    return(
      div(class = "alert alert-info",

```

```

    style = "margin-top: 15px;",
    icon("info-circle"),
    "O teste de Log-Rank requer duas ou mais curvas para comparação.")
  )
}

# Realiza o teste de Log-Rank
logrank_test <- survdiff(Surv(tempo, DESFECHO) ~ Grupo, data = dados)

# Extrai a estatística e o p-valor
chisq_stat <- logrank_test$chisq
p_value <- 1 - pchisq(logrank_test$chisq, length(logrank_test$n) - 1)

# Formata o p-valor para melhor visualização
p_value_formatted <- if (p_value < 0.001) {
  "< 0.001"
} else {
  format(round(p_value, 3), nsmall = 3)
}

# Cria o output em HTML
tagList(
  hr(),
  h4("Resultado do Teste de Log-Rank")),
  p(class = "justified-text",
    "O teste de Log-Rank compara as curvas de sobrevivência de dois ou mais grupos.
    Um p-valor pequeno (geralmente < 0.05) sugere que há uma diferença estatisticamente
    significativa entre as curvas."),
  p(HTML(paste0("<b>Estatística Qui-quadrado:</b> ", round(chisq_stat, 3)))),
  p(HTML(paste0("<b>P-valor:</b> ", p_value_formatted)))
)
})

# ===== FIM DA MODIFICAÇÃO =====

#####

## Tabela Kaplan-Meier

output$tabelas_por_grupo <- renderUI({
  req(input$grupo_cid_2)

  dados <- dados_filtrados_km() |> arrange(Grupo)
  fit <- survfit(Surv(tempo, DESFECHO) ~ Grupo, data = dados)
  grupos <- unique(dados$Grupo)

  tagList(
    shinyWidgets::pickerInput(
      inputId = "grupos_selecionados",
      label = "Selecione os grupos para exibir as tabelas:",
      choices = grupos,
      selected = grupos[1:min(3, length(grupos))],
      multiple = TRUE,
      options = list(`actions-box` = TRUE, `deselect-all-text` = "Nenhuma", `select-all-text` = "Todos")
    )
  )
})

```

```

    ),
    withSpinner(uiOutput("tabelas_selecionadas"))
  )
})

output$tabelas_selecionadas <- renderUI({
  req(input$grupos_selecionados)

  dados <- dados_filtrados_km() |> arrange(Grupo)
  fit <- survfit(Surv(tempo, DESFECHO) ~ Grupo, data = dados)

  tabelas <- map(input$grupos_selecionados, ~ {

    grupo_selecionado <- .x
    indice_grupo <- which(names(fit$strata) == grupo_selecionado)

    if (length(indice_grupo) == 0) {
      indice_grupo <- which(grepl(grupo_selecionado, names(fit$strata)))
    }

    req(length(indice_grupo) > 0)

    inicio <- ifelse(indice_grupo == 1, 1, sum(fit$strata[1:(indice_grupo-1)]) + 1)
    fim <- sum(fit$strata[1:indice_grupo])

    dados_grupo <- list(
      time = fit$time[inicio:fim],
      surv = fit$surv[inicio:fim],
      std.err = fit$std.err[inicio:fim],
      lower = fit$lower[inicio:fim],
      upper = fit$upper[inicio:fim],
      n.event = fit$n.event[inicio:fim],
      n.censor = fit$n.censor[inicio:fim],
      n.risk = fit$n.risk[inicio:fim]
    )

    tabela <- data.frame(
      tempo = dados_grupo$time,
      Sobrevivência = dados_grupo$surv,
      Erro_Padrão = dados_grupo$std.err,
      IC_Inferior = dados_grupo$lower,
      IC_Superior = dados_grupo$upper,
      Eventos = dados_grupo$n.event,
      Censuras = dados_grupo$n.censor,
      Em_Risco = dados_grupo$n.risk
    )

    names(tabela)[1] <- janela_tempo()

    tagList(
      h4(paste("Grupo:", grupo_selecionado)),
      renderDT({
        datatable(

```

```

    tabela,
    options = list(
      pageLength = 10,
      lengthChange = FALSE,
      language = list(
        url = '//cdn.datatables.net/plug-ins/1.10.11/i18n/Portuguese-Brasil.json'
      ),
      ordering = FALSE,
      searching = FALSE,
      info = FALSE
    ),
    rownames = FALSE
  ) |>
  formatRound(columns = c('Sobrevivência', 'Erro_Padrão', 'IC_Inferior', 'IC_Superior'),
    digits = 4)
  },
  hr()
)
})

do.call(tagList, tabelas)
})

#####

janela_tempo2 <- reactive({
  case_when(
    input$len_tempo2 == "MESES" ~ "Tempo (meses)",
    input$len_tempo2 == "TRI" ~ "Tempo (trimestres)",
    input$len_tempo2 == "ANO" ~ "Tempo (anos)"
  )
})

output$alert_box3 <- renderUI({
  if(length(input$grupo_cid_3) < 1) {
    div(class = "alert alert-warning",
      icon("exclamation-triangle"),
      "Selecione, pelo menos, uma variável para visualizar o gráfico.")
  }
})

## Gráfico função de risco

dados_filtrados_hazard <- reactive({
  # ALTERADO: Usa dados_cancer_reativo() em vez de dados_cancer
  dados_cancer_reativo() |>
  filter(TOPOGRUP_GRUPO %in% input$grupo_cid_3) |>
  select(tempo = paste0("TEMPO_OBS_", input$Tempo_int2, "_", input$len_tempo2),
    Grupo = input$hazard_variable,
    DESFECHO
  ) |>
  mutate(Grupo = as.factor(Grupo)) |>
  as.data.frame()
})

```

```

})

output$hazard_plot <- renderHighchart({

  req(input$grupo_cid_3)

  dados <- dados_filtrados_hazard()

  categorias <- levels(dados$Grupo)
  lista_risco <- list()

  for(cat in categorias) {
    dados_cat <- dados |> filter(Grupo == cat)

    # >>>> INÍCIO DA CORREÇÃO <<<<
    # O bloco tryCatch impede que o app feche se a função muhaz() falhar.
    estimativa <- tryCatch({
      muhaz(
        times = as.numeric(dados_cat$tempo),
        delta = dados_cat$DESFECHO,
        min.time = min(dados_cat$tempo),
        max.time = max(dados_cat$tempo)
      )
    }, error = function(e) {
      # Se ocorrer um erro, retorna NULL para que possamos ignorar este grupo.
      NULL
    })

    # Pula para a próxima iteração do loop se a estimativa falhou
    if (is.null(estimativa)) {
      next
    }

    # >>>> FIM DA CORREÇÃO <<<<

    lista_risco[[cat]] <- data.frame(
      tempo = estimativa$est.grid,
      risco = estimativa$haz.est,
      categoria = cat
    )
  }

  # Se nenhuma estimativa funcionou, mostra um gráfico vazio
  validate(
    need(length(lista_risco) > 0, "Não foi possível estimar a função de risco para a escala de tempo ")
  )

  df_plot <- bind_rows(lista_risco)

  hchart(
    df_plot,
    type = "line",
    hcaes(x = tempo, y = risco, group = categoria),
    marker = list(enabled = FALSE)
  )

```

```

) |>
  hc_title(text = "Função de Risco") |>
  hc_xAxis(title = list(text = paste0(janela_tempo2()))) |>
  hc_yAxis(title = list(text = "Taxa de Risco Instantânea")) |>
  hc_tooltip(
    headerFormat = "<b>{point.series.name}</b><br>",
    pointFormat = paste0(janela_tempo2(), ": {point.x:.2f} <br> Risco: {point.y:.4f}")
  ) |>
  hc_legend(align = "center", verticalAlign = "bottom", layout = "horizontal")
})

#####

#### aba Modelo de Cox

#####

# Alerta para o usuário selecionar as variáveis necessárias
output$alert_box4 <- renderUI({
  if(length(input$grupo_cid_4) < 1 || length(input$cox_variables) < 1) {
    div(class = "alert alert-warning",
        icon("exclamation-triangle"),
        "Selecione, pelo menos, um grupo de topografia e uma variável para ajustar o modelo.")
  }
})

# Filtra os dados de forma reativa para o modelo de Cox
dados_filtrados_cox <- reactive({
  req(input$grupo_cid_4, input$cox_variables)

  # ALTERADO: Usa dados_cancer_reativo() em vez de dados_cancer
  dados_cancer_reativo() |>
  filter(TOPOGRUP_GRUPO %in% input$grupo_cid_4) |>
  select(
    tempo = paste0("TEMPO_OBS_", input$Tempo_int3, "_", input$len_tempo3),
    DESFECHO,
    all_of(input$cox_variables)
  ) |>
  mutate(tempo = as.numeric(tempo)) |>
  na.omit()
})

# Reactive para ajustar o modelo de Cox (para ser reutilizado)
cox_model_fit <- reactive({
  req(input$cox_variables)

  dados <- dados_filtrados_cox()

  validate(
    need(nrow(dados) > 0, "")
  )

  formula_str <- paste("Surv(tempo, DESFECHO) ~", paste(input$cox_variables, collapse = " + "))

```

```

cox_formula <- as.formula(formula_str)

coxph(cox_formula, data = dados)
})

# Tabela de resultados agora usa o modelo do reactive `cox_model_fit`
output$cox_summary_table <- renderDT({

  validate(
    need(nrow(dados_filtrados_cox()) > 0, "Não há dados suficientes para as seleções de filtros aplicadas")
  )

  cox_model <- cox_model_fit()

  summary_df <- broom::tidy(cox_model, exponentiate = TRUE, conf.int = TRUE) |>
    select(
      Variável = term,
      `Hazard Ratio (HR)` = estimate,
      `IC Inferior` = conf.low,
      `IC Superior` = conf.high,
      `Valor-p` = p.value
    )

  datatable(
    summary_df,
    options = list(pageLength = 10, lengthChange = FALSE, searching = FALSE, info = FALSE,
      language = list(url = '//cdn.datatables.net/plug-ins/1.10.11/i18n/Portuguese-Brasil.js')
    ), rownames = FALSE
  ) |>
    formatRound(columns = c('Hazard Ratio (HR)', 'IC Inferior', 'IC Superior'), digits = 3) |>
    formatRound(columns = c('Valor-p'), digits = 2)
})

# Forest Plot do Modelo de Cox
output$cox_forest_plot <- renderPlot({

  validate(
    need(nrow(dados_filtrados_cox()) > 0, "Não há dados suficientes para gerar o forest plot.")
  )

  ggforest(
    model = cox_model_fit(),
    data = dados_filtrados_cox()
  )
})

# Lógica para os resíduos de Schoenfeld

# 1. Gera o seletor de variáveis dinamicamente
output$schoenfeld_variable_selector_ui <- renderUI({
  req(input$cox_variables)

```



```

tagList(
  h3("Teste de Pressupostos (Resíduos de Schoenfeld)"),
  p("Este teste verifica a premissa de riscos proporcionais do modelo de Cox.  

  Se a linha suavizada no gráfico for aproximadamente horizontal e o p-valor for maior que 0.05,  

  a premissa é considerada atendida para aquela variável."),

  pickerInput(
    inputId = "schoenfeld_vars",
    label = "Selecione as variáveis para visualizar os resíduos:",
    choices = input$cox_variables,
    selected = input$cox_variables,
    multiple = TRUE,
    options = list(`actions-box` = TRUE, `deselect-all-text` = "Nenhuma", `select-all-text` = "Todas")
  )
)
})

# Trocado eventReactive por reactive para reagir a TODAS as alterações, incluindo tempo.
terms_to_plot_reactive <- reactive({
  req(cox_model_fit(), input$schoenfeld_vars)

  schoenfeld_test <- cox.zph(cox_model_fit())
  all_terms <- rownames(schoenfeld_test$table)

  terms_to_plot <- unlist(sapply(input$schoenfeld_vars, function(var) {
    grep(paste0("^", var), all_terms, value = TRUE)
  }))

  list(schoenfeld_test = schoenfeld_test, terms = terms_to_plot)
})

# 2. Gera os placeholders para os gráficos
output$schoenfeld_plots_ui <- renderUI({
  plot_data <- terms_to_plot_reactive()
  req(plot_data$terms)

  map(plot_data$terms, ~ {
    plot_id <- gsub("[^[:alnum:]]", "", .x)
    plotOutput(outputId = paste0("schoenfeld_plot_", plot_id), height = "400px")
  })
})

# 3. Renderiza cada gráfico individualmente
observe({
  plot_data <- terms_to_plot_reactive()
  req(plot_data$terms)

  schoenfeld_test <- plot_data$schoenfeld_test

  for (term in plot_data$terms) {
    local({
      my_term <- term
      plot_id <- gsub("[^[:alnum:]]", "", my_term)
    })
  }
})

```

```

output[[paste0("schoenfeld_plot_", plot_id)]] <- renderPlot({
  plot_list <- ggcoxzph(schoenfeld_test, var = my_term, point.alpha = 0.5)

  plot_list[[1]] +
    labs(
      title = paste("Resíduos de Schoenfeld para:", my_term),
      subtitle = paste("Teste de Proporcionalidade, p-valor:", round(schoenfeld_test$table[my_t
    ) +
    theme_bw()
  })
})
}
})

# PDF da documentação
observeEvent(input$generate, {
  output$pdfview <- renderUI({
    tags$iframe(style = "height:600px; width:100%", src = "documentacao.pdf")
  })
})

# Atualizando a seleção para evitar bugs para CID's de sexo exclusivo

## KM
observe({
  # ATENÇÃO: Adicione "IDADE ESTRATIFICADA" às opções aqui
  opcoes_base <- c("Faixa etária" = "FAIXAETAR", "Sexo" = "SEXO",
    "Idade (ponto de corte)" = "IDADE ESTRATIFICADA",
    "Estágio clínico" = "GRUPO_EC", "Tratamento" = "TRATAMENTO")

  req(input$grupo_cid_2)

  grupos_especificos <- c("C51-C58 Órgãos genitais femininos",
    "C60-C63 Órgãos genitais masculinos")

  opcoes_finais <- opcoes_base

  # Se APENAS um grupo de sexo específico for selecionado, remove a opção "Sexo"
  if (any(grupos_especificos %in% input$grupo_cid_2) && length(input$grupo_cid_2) == 1) {
    opcoes_finais <- opcoes_base[names(opcoes_base) != "Sexo"]

    # Se "Sexo" estava selecionado, muda para outra opção para evitar erro
    if (!is.null(input$km_variable) && input$km_variable == "SEXO") {
      updatePickerInput(session, "km_variable", selected = "IDADE ESTRATIFICADA")
    }
  }

  updatePickerInput(session, "km_variable", choices = opcoes_finais)
})

## Hazard
observeEvent(input$grupo_cid_3, {

```

```

# ===== INÍCIO DA CORREÇÃO =====
opcoes <- c("Faixa etária" = "FAIXAETAR", "Sexo" = "SEXO",
           "Idade (ponto de corte)" = "IDADE_ESTRATIFICADA",
           "Estágio clínico" = "GRUPO_EC", "Tratamento" = "TRATAMENTO")
# ===== FIM DA CORREÇÃO =====

grupos_especificos <- c("C51-C58 Órgãos genitais femininos",
                       "C60-C63 Órgãos genitais masculinos")

algun <- any(grupos_especificos %in% input$grupo_cid_3)

if ((algun & length(input$grupo_cid_3) == 1)) {
  opcoes <- opcoes[opcoes != "SEXO"]

  selecionado_atual <- input$hazard_variable
  if (selecionado_atual == "SEXO") {
    updateSelectInput(session, "hazard_variable", selected = "FAIXAETAR")
  }
}

updateSelectInput(session, "hazard_variable", choices = opcoes)
})
}

```

4.0.4 app.R

```

## app.R

source("ui.R")
source("server.R")

shinyApp(ui = ui, server = server)

```